UFR de mathématique et d'informatique
Université de Strasbourg

CMI d'informatique
Cursus Master en Ingénierie en Informatique, Systèmes et Réseaux

Travail d'Étude et de Recherche (TER)

Jeronimo Herdoïza
jeronimo.herdoiza@etu.unistra.fr

Thread Scheduling for Transformers

Sunday 3rd May, 2026

Research work supervised by

Cedric Bastoul
bastoul@unistra.fr

Contents

Contents **2**

1 Introduction **3**

2 Background **3**

3 State of the art **3**

3.1 Thread Scheduling in Parallel Computing 3

3.2 Work Sharing 3

3.3 Work Stealing 3

3.4 Advanced Scheduling Variants 4

4 Implementation **5**

5 Methodology **5**

6 Discussion **5**

7 Conclusion **5**

8 References **5**

1 Introduction

2 Background

3 State of the art

3.1 Thread Scheduling in Parallel Computing

Thread scheduling has been a fundamental problem im parallel computing since several decades: given a set of computations and a fixed number of CPU cores, the scheduler has to decide which thread runs on which core at which time. As of today, where multicore architecures have become the norm, these decisions have a critical impact on load balance, cache utilization, memory locality, and synchronization overhead. To address this problem, two main families of scheduling policies have been found to be effective: *work sharing* and *work stealing*.

3.2 Work Sharing

With *work sharing* when new work is created, the scheduler assigns the tasks among the available worker threads. This approach is rather simple but can lead to poor load balance when the cost of iterations varies significantly, this is due to the fact that the work distribution is set before the loop begins and cannot be adjusted at runtime.

OpenMP is the most widely used API in this domain [MN24]. It provides several clauses that aim to avoid this problem and improve load balance. For instance, the `dynamic` clause allows the scheduler to assign iterations to threads dynamically as they become available. The `guided` clause works in a similar way but executes large chunks of iterations at the beginning of the loop and decrease the size of these chunks as the loop progresses. Finally, the `static` clause works the way described above, in most implementations of OpenMP, this is the default scheduling policy.

3.3 Work Stealing

Work stealing was formalized by Blumofe and Leiserson [BL99] in 1999. They take a different approach than work sharing: rather than pushing work from a central source, idle threads will pull tasks from other busy threads. This means that all threads have their own double-ended queue (deque). Inside each thread, tasks first treated in a LIFO order to preserve locality and cache reuse, on the other hand, thieves will come and steal from the oposite end of the deque (FIFO).

Blumofe and Leiserson proved that on P processors, the expected execution time of a fully strict multithreaded computation under work stealing satisfies:

$$T_P \leq \frac{T_1}{P} + O(T_\infty)$$

where T_1 is the serial execution time and T_∞ is the critical-path length with unlimited processors. Space usage is bounded at $S_1 P$, and expected total communication at $O(PT_\infty(1+n_d)S_{\max})$, where

$S_{\max}$ is the size of the largest activation record and n_d is the maximum thread synchronization depth. This communication bound formally justifies the intuition that work-stealing schedulers are more efficient than work-sharing counterparts: stolen tasks are coarse-grained and steals are rare in balanced workloads [BL99].

The work by Blumofe and Leiserson [BL99] made work stealing widely adopted in parallel programming. For instance OpenMP implemented work stealing in its task clause. However, this implementation is not standardized in the API and varies significantly among different runtimes, making it difficult to reason about or tune for a specific workload.

3.4 Advanced Scheduling Variants

Several variants of work stealing have been proposed in the literature because of the limitations of the original scheme. Especially in the context of Non Uniform Memory Access (NUMA) architectures, such as the ones studied in this work, where stealing from a remote thread can be costly. This has motivated the design of NUMA-aware work stealing and other scheduling strategies.

Cost-Aware Work Stealing (COWS). Mishra and Nandivada [MN24] identify three fundamental shortcomings when work stealing is applied to parallel-for loops:

- not all iterations take the same time, this means that stealing a fixed number of iterations will not equalize the load,
- selecting a suitable victim is non-trivial, and selecting one randomly performs poorly,
- the per-thread deque required by traditional work stealing introduces overhead that exceeds its benefit for moderately irregular loops.

Their scheme addresses all three simultaneously.

COWS proposes two variants. Work Stealing based on Remaining Iterations (*WSRI*) tracks the number of remaining iterations for each thread in a lightweight shared scalar, replacing the deque entirely, and uses this information to select the most loaded victim. Work Stealing based on Remaining Workload (*WSRW*) uses a compile-time analysis to emit application-specific instrumentation code, enabling the runtime to estimate the remaining computational cost of each thread rather than merely its iteration count. In 225 out of 275 tested configurations spanning 7 benchmarks, 5 datasets, and 2 hardware platforms, COWS outperforms the best available OpenMP scheduling scheme, with clear advantages for power-law graph workloads where static and dynamic schedules both fail [MN24].

NUMA-Aware and Distance-Aware Work Stealing. It's well known in systems programming that memory accesses are costly. On NUMA architectures, a naive implementation of work stealing can lead to significant performance issues due to this issue. Indeed, when a thread steals a task from a remote NUMA node, it may move the task far from its data and cause repeated remote memory accesses for the stolen task's entire execution.

Applied to Cholesky factorization and symmetric matrix inversion on a large NUMA system, this approach achieves up to a twofold speedup in flop rate over non-discriminating policies and matches or exceeds the performance of state-of-the-art commercial and open-source HPC libraries [AOML+15]. The key insight is that on modern hardware, the scheduler must treat the memory topology of the machine as a first-class input to its decision-making, not an afterthought.

Al-Omairy et al. [AOML+15] address this with a *distance-aware* scheduling policy built on the OmpSs task-based programming model. The key insight for their scheduler must treat the memory topology of the machine as a critical input to its decision-making, not as a secondary concern. They approach the scheduling in a topology-aware manner. For instance, a thief will prefer to steal from its nearest NUMA neighbor, making the decision to go for more distant nodes exclusively when local work is exhausted. When applied to Cholesky factorization and symmetric matrix inversion on a large NUMA system, the results show up to a twofold speedup, matching or exceeding the performance of state-of-the-art commercial and open-source HPC libraries [AOML+15].

4 Implementation

Work sharing with OpenMP was studied in depth in the P̈arallel Programmingc̈ourse during the automn semester. In this TER, the main focus for the research and implementation will be on work stealing.

5 Methodology

6 Discussion

7 Conclusion

8 References

[AOML+15] Rabab Al-Omairy, Guillermo Miranda, Hatem Ltaief, Rosa M. Badia, Xavier Martorell, Jesus Labarta, and David Keyes. Dense matrix computations on NUMA architectures with distance-aware work stealing. *Supercomputing Frontiers and Innovations*, 2(1):49–71, 2015.

[BL99] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, September 1999.

[MN24] Prasoon Mishra and V. Krishna Nandivada. COWS for high performance: Cost aware work stealing for irregular parallel loop. *ACM Transactions on Architecture and Code Optimization*, 21(1):1–26, January 2024.